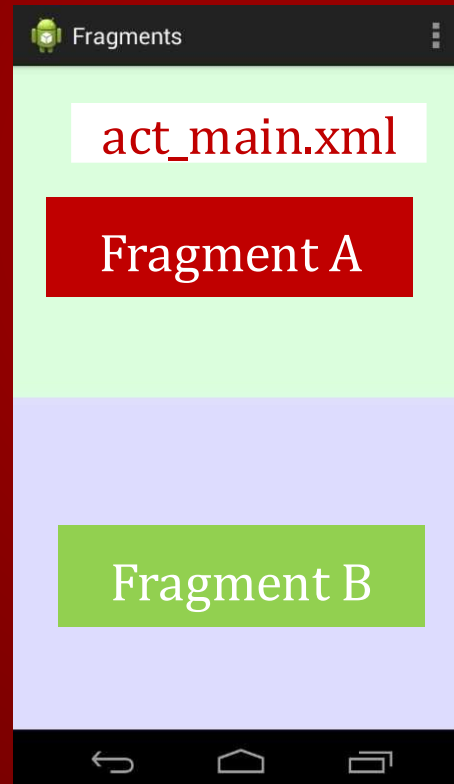
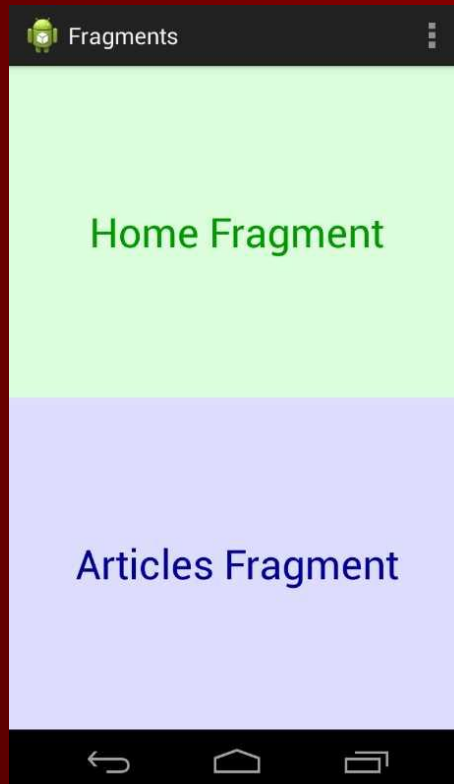


ANDROID FRAGMENTS

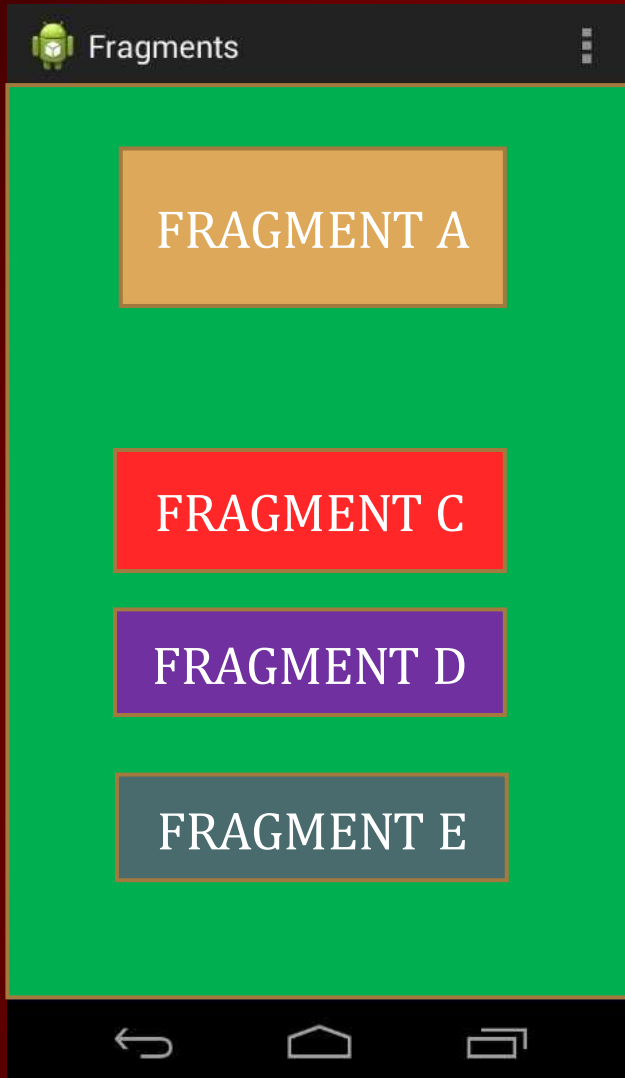
Prof.K.Sathiyamurthy, DCSE, PTU

ANDROID FRAGMENTS

- ❑ Fragments are found inside activity
- ❑ One Activity can contain multiple Fragments



Fragment A – Add
Fragment B - Add



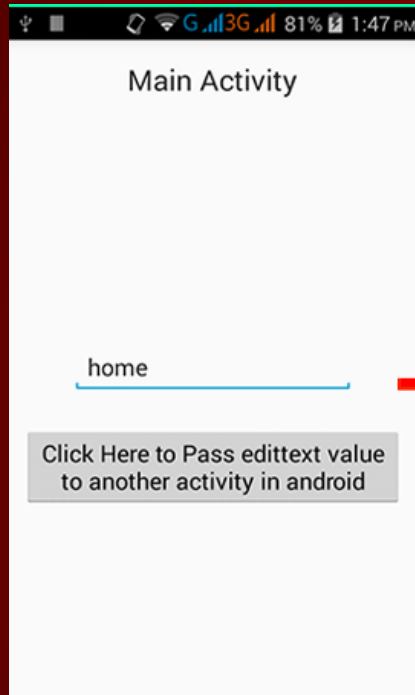
Fragment A – Add
Remove B - Remove
Fragment C – Add
Fragment D - Add
Fragment E – Add

- Acts as a small chunk of UI
- All Fragments are integral part of the `main_activity.java` (i.e its child elements)

FRAGMENTS

- ❑ Small Chunk of UI
- ❑ Has its own life cycle
- ❑ Can process its own events
- ❑ Can be added/removed while parent activity is running
- ❑ Introduced in API 11 – Honeycomb with the introduction of tablets
- ❑ Backward compatible upto API 7
- ❑ Act as a mini Activity

Activity



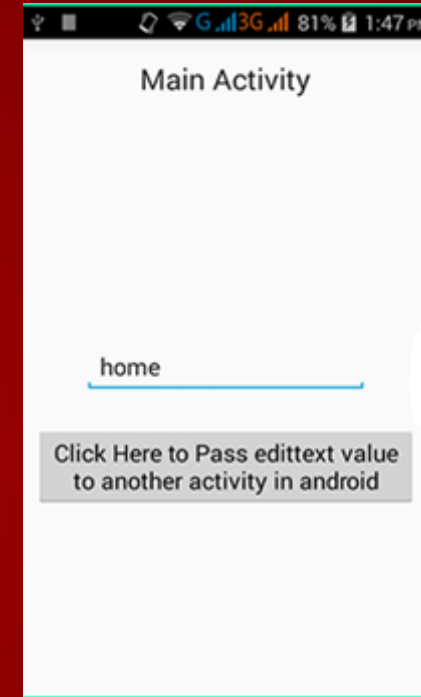
Main Activity



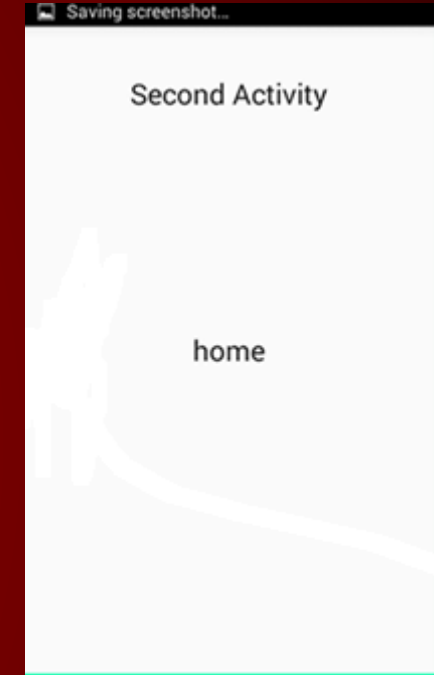
Second Activity

- ❑ Need two activities to display two kinds of layouts

Activity + Fragment



Main Activity



- ❑ Need only one Main activity to display two kinds of layouts

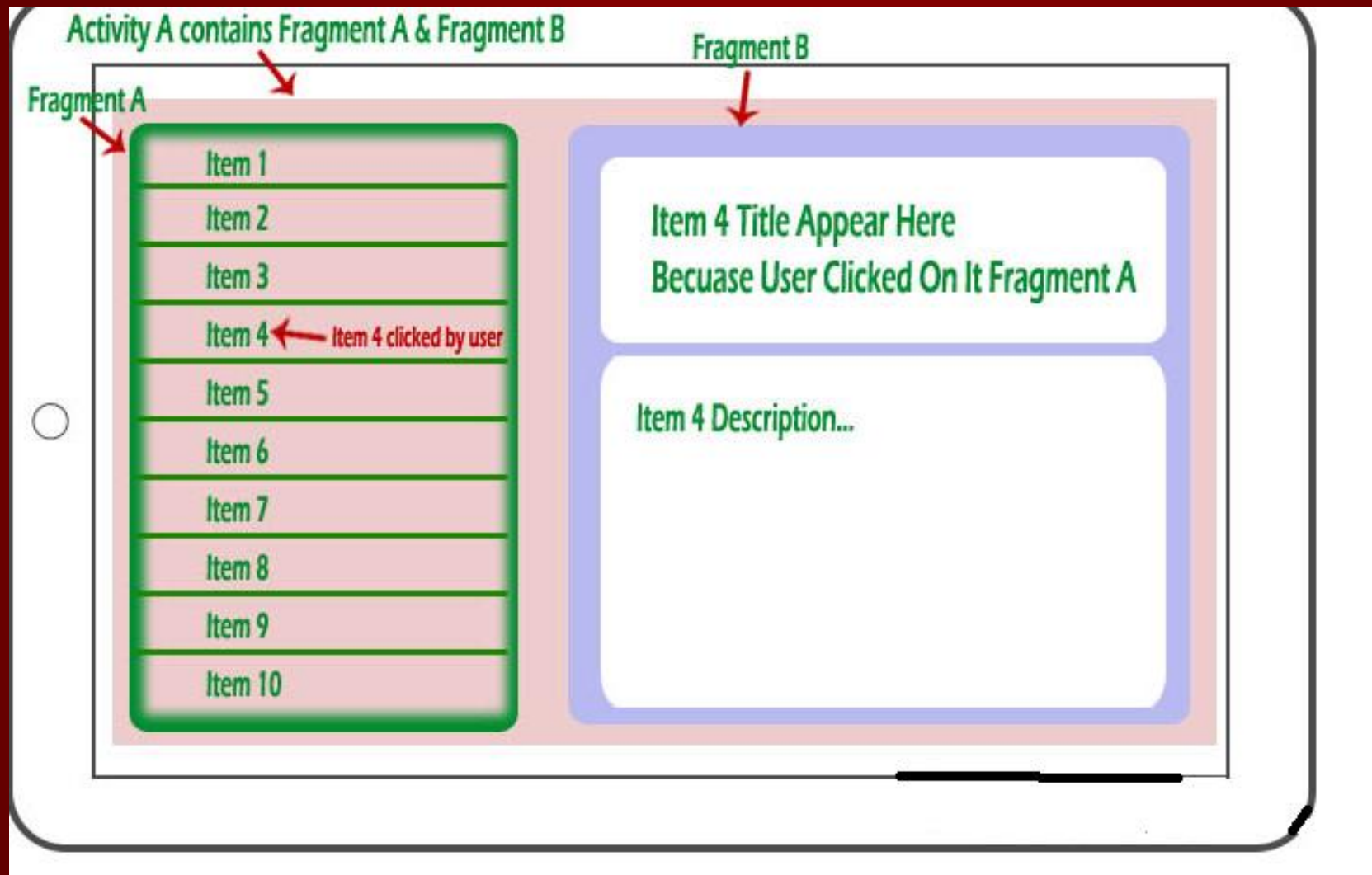


Image Gallery App



Main Activity

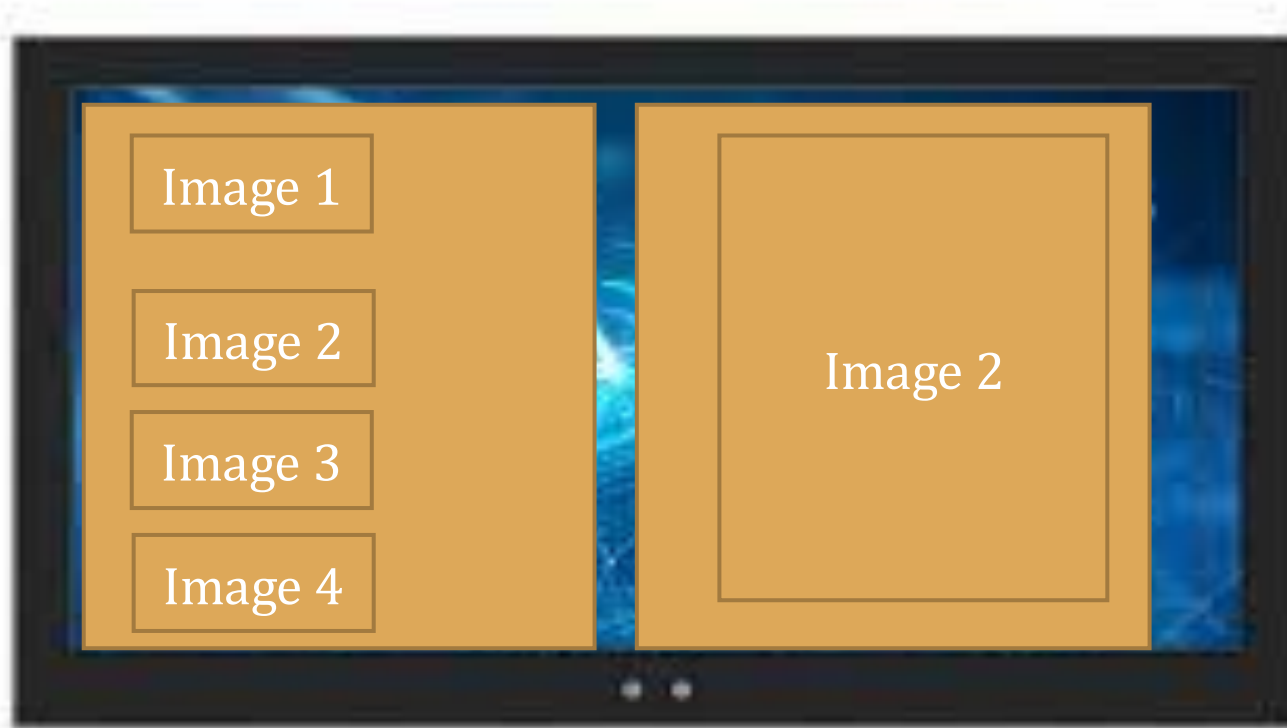


Second Activity

IMAGE GALLERY APP - TABLET

Major Advantage of Fragments
introduced with tablets

Enabling developers to effectively
use space in the tablets



Fragment A-List

Fragment B- Detailed View

Main Activity

Why Fragments?

- ❑ Combine several fragments in single activity
- ❑ Reuse same fragment across several activities
- ❑ Make better use of larger screen space in Tablets
- ❑ Support different layouts on portrait and landscape modes

Adding Fragments to Activity

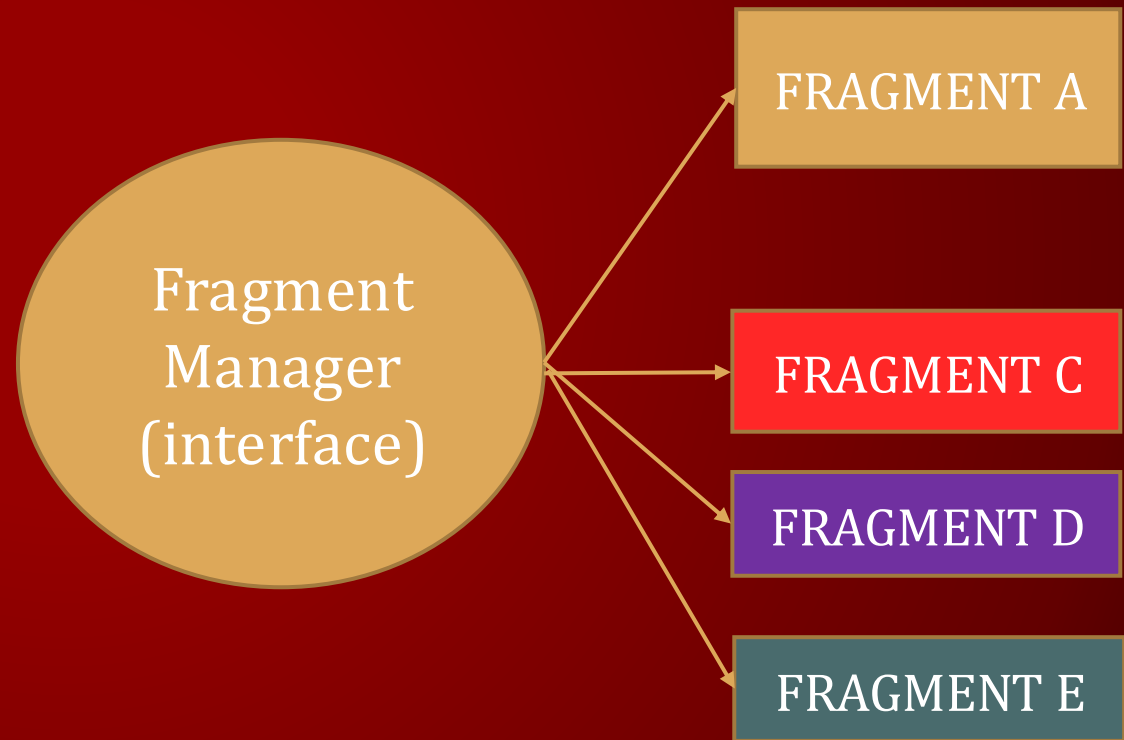
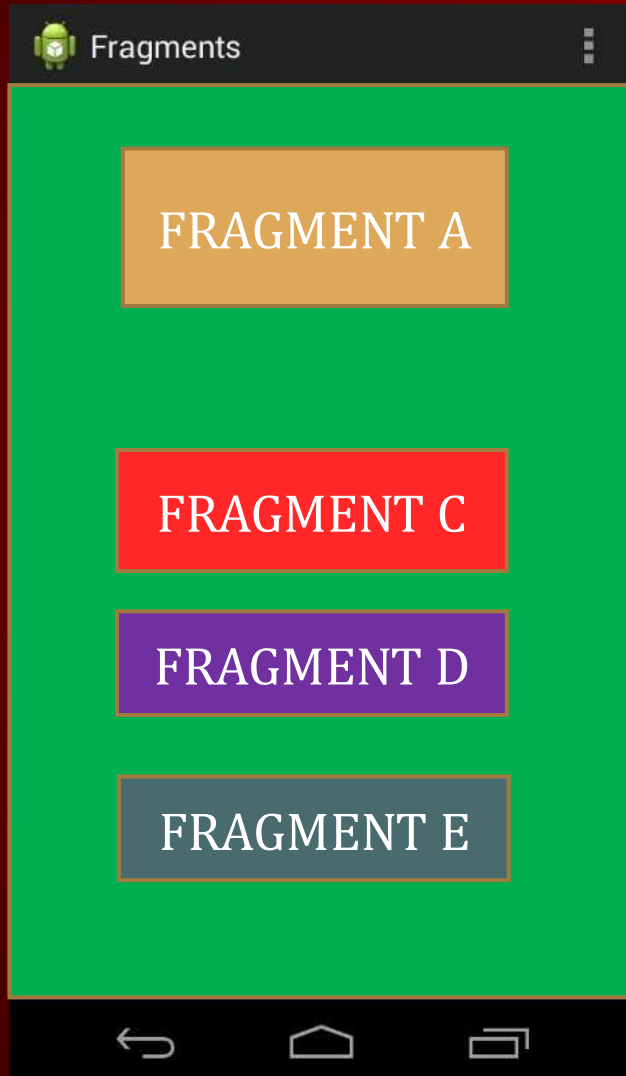
- ❑ Two Approaches
 - ❑ XML (less preferred)
 - ❑ Java / Programmatically (More Preferred)

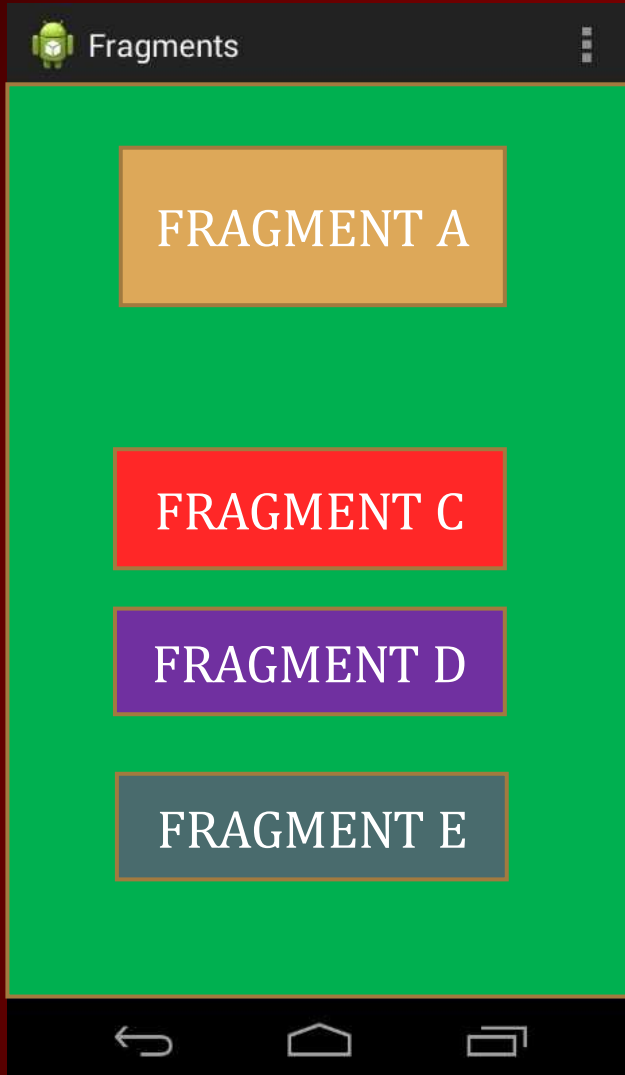
1. Steps to create Fragment by XML

- ❑ Create a subclass of Fragment
 - ❑ HelloFragment.java
- ❑ Create a layout for fragment
 - ❑ fragment hello.xml
- ❑ Link layout with Fagment subclass
 - ❑ override onCreateView()
- ❑ Place the Fragment inside an Activity
 - ❑ <fragment> inside activity_main.xml

Fragment Manager

- Interface to interact with fragment objects inside the Activity
- Handles all Fragments inside one activity





Fragment Transaction

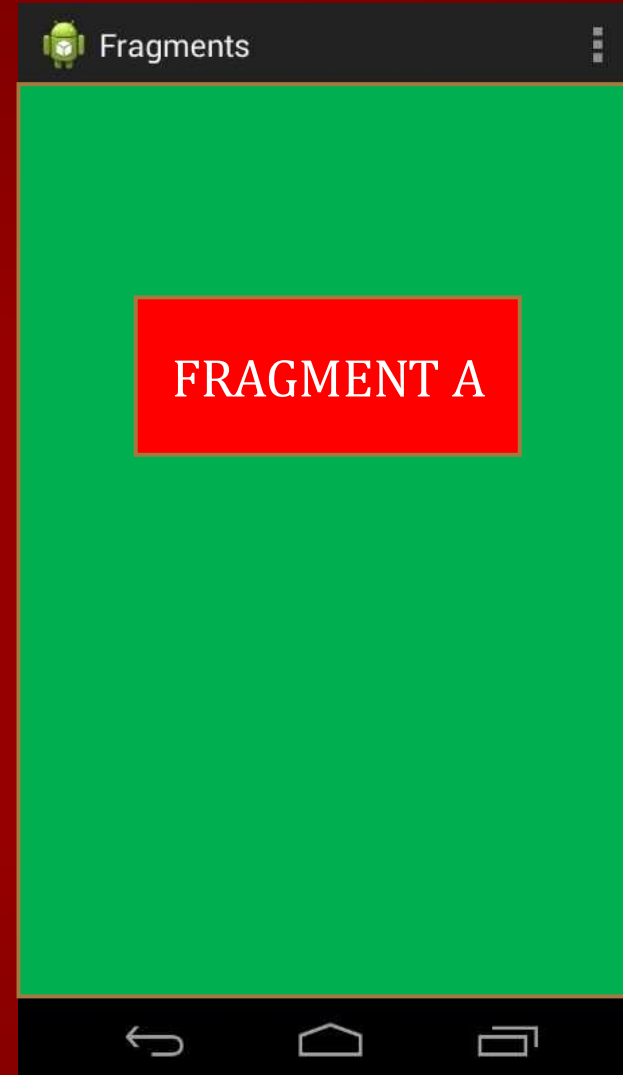
- Fragment Transaction takes place
Fragment A – Add
Remove B - Remove
Fragment C – Add
Fragment D - Add
Fragment E – Add
- Addition and removal of fragments are known as fragment Transactions
- Takes place in Main Activity with help of Fragment Transaction class.
- Fragment Transaction- API for performing a set of Fragment operations such as add, remove, replace.

2.Steps to create Fragments by Java/Programmatically

- ❑ Create a subclass of Fragment
 - ❑ HelloFragment.java
- ❑ Create a layout for fragment
 - ❑ fragment hello.xml
- ❑ Link layout with Fagment subclass
 - ❑ override onCreateView()
- ❑ Place the Fragment inside an Activity
 - ❑ Initialize Fragment Manager
 - ❑ Initialize Fragment Transaction
 - ❑ Start add/remove/replace operation
 - ❑ Commit the transaction

FRAGMENTS LIFECYCLE

FRAGMENTS LIFECYCLE



FRAGMENTS TRANSACTIONS

- ❑ Transaction Types
 - ❑ Add
 - ❑ Remove
 - ❑ Replace
 - ❑ Attach
 - ❑ Detach
 - ❑ Show
 - ❑ Hide

FRAGMENTS TRANSACTIONS

❑ ADD

- Create a new Fragment
- Get the id of the container layout
- Use `add( )` to add the fragment
 - The Fragment is added to the container layout

❑ Remove

- Get the reference to the fragment to be removed
- Use `remove()` to remove the fragment
 - The fragment is destroyed
 - Detached from the host activity

FRAGMENTS TRANSACTIONS

- ❑ Replace
 - Get the id of the container layout
 - Get a reference to the new Fragment
 - Use `replace()` to replace the content fragment of the container layout
 - The new Fragment fills the container
 - The old fragment if any is destroyed

FRAGMENTS TRANSACTIONS

❑ Detach

- Get the reference to the fragment
- Use detach() to detach the fragment from Activity UI
 - The fragment's view is destroyed
 - The Fragment and its UI is detached from the host Activity
(object is present and it is not visible to the user)

❑ Attach

- Get the reference to the Detached fragment
- Use attach() to reattach the fragment
 - The fragment's view is recreated
 - The Fragment is re-attached to the activity and is visible

FRAGMENTS TRANSACTIONS

❑ Hide

- Get the reference to the fragment to hide
- Use `hide()` to hide the fragment from Activity UI
 - The fragment's view becomes invisible
 - The Fragment continue to be in Resumed state

❑ Show

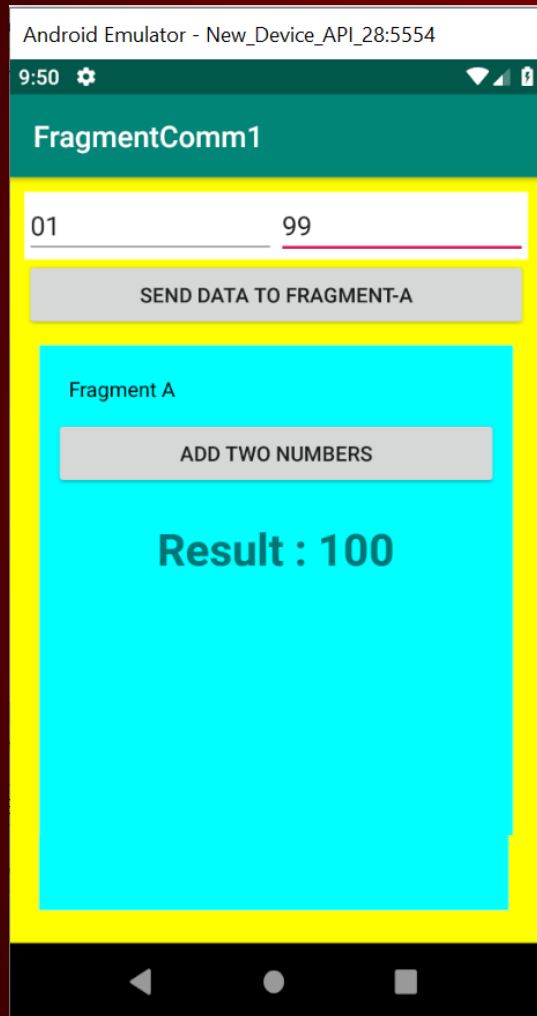
- Get the reference to the Hidden fragment
- Use `show()` to make it visible
 - The view of the fragment is now visible
 - Remains in Resumed state as it was earlier

- Hide and show does not affect the states of the transactions

COMMUNICATION

- ❖ **SENDING DATA TO A FRAGMENT FROM ACTIVITY**
- ❖ **SENDING DATA TO ACTIVITY FROM FRAGMENT**
- ❖ **INTER FRAGMENT COMMUNICATION**

❖ 1. SENDING DATA TO A FRAGMENT FROM ACTIVITY



- ❑ Using Bundle Object
 - ❑ Stores key/value pair
- ❑ Using Fragment Object

2. SENDING DATA TO AN ACTIVITY FROM FRAGMENT

Activity implements
MyListener

```
@override  
Public void add(int x, int y){  
    //add x +y  
    //show result  
}
```

Fragment

```
activity.add(x,y)
```

```
interface myListener  
public void add(int num1,int num2)
```

```
graph LR; subgraph Activity; direction TB; A1[Activity implements MyListener]; A2["@override  
Public void add(int x, int y){  
    //add x +y  
    //show result  
}"]; end; subgraph Fragment; direction TB; F1[Fragment  
activity.add(x,y)]; end; I1["interface myListener  
public void add(int num1,int num2)"]; A1 --> I1; F1 --> A2;
```

MainActivity extends Activity implements MyListener{

@override

Public void add(int num1, int num2) {

int result = num1 + num2;

txvResult.setText("Result:" + result);

}

}

Class FragmentA extends Fragment{

Private void sendData{

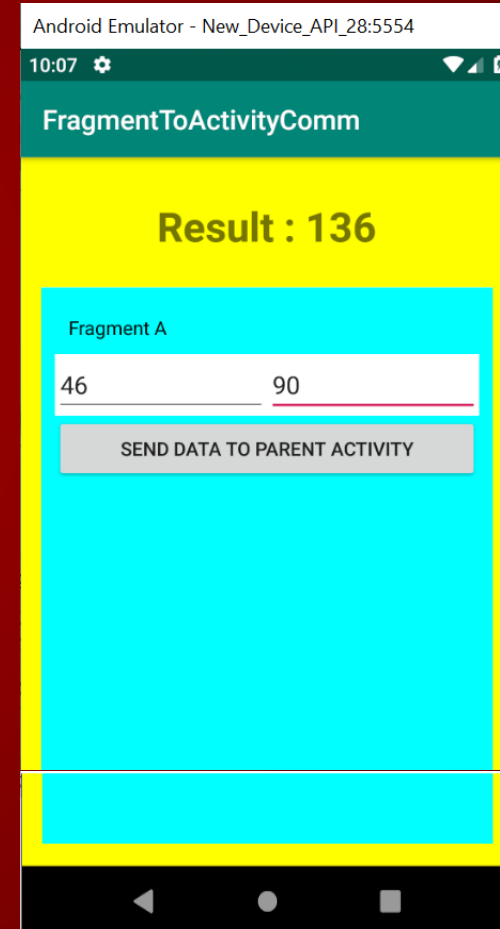
MyListener listener=(MyListener) getActivity();

Listener.add(23,17);

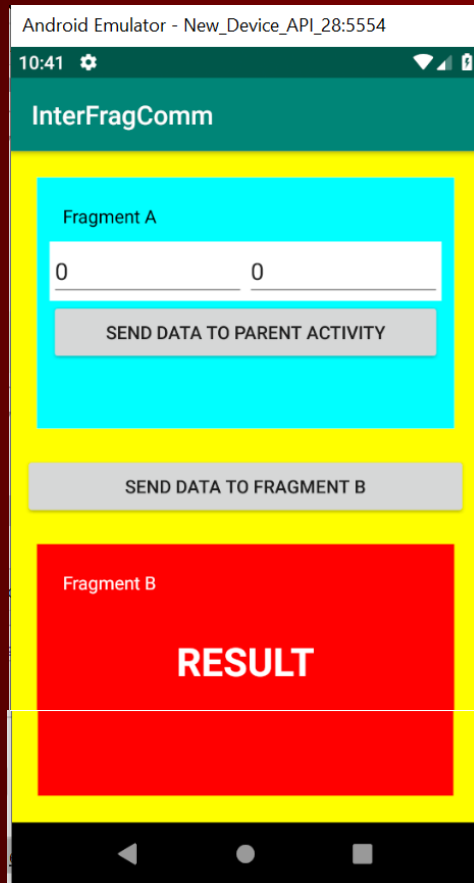
}

}

```
Interface MyListener{  
    public void add(int num1, int num2);  
}
```



❖ 3. SENDING DATA FROM ONE FRAGMENT TO ANOTHER



Take input in Fragment A

- Get FirstNumber
- Get SecondNumber

Pass the Data to FragmentB

- Add Two numbers(FirstNumber + SecondNumber)
- Display the Result

❖ COMMUNICATION BETWEEN TWO FRAGMENTS

Activity implements MyListener

Fragment A

activity.add(x,y)

@Override

Public void add(int x, int y){
fragB.addTwoNumbers(x,y) }

Activity

Fragment B

```
Public void addTwoNumbers(int a, int b) {  
int result = a+b;  
}
```

interface myListener

```
public void add(int num1, int num2);
```

- Pass data from fragmentA to Activity
- Pass data from Activity to FragmentB
- Add method in FragmentA
- call add method in activity class to call fragB.addTwoNumbers(x,y)
- It executes addTwoNumbers in FragmentB

* not communicating directly but creating a communication gap between two fragments

❖ STEPS FOR SENDING DATA FROM ONE FRAGMENT TO ANOTHER

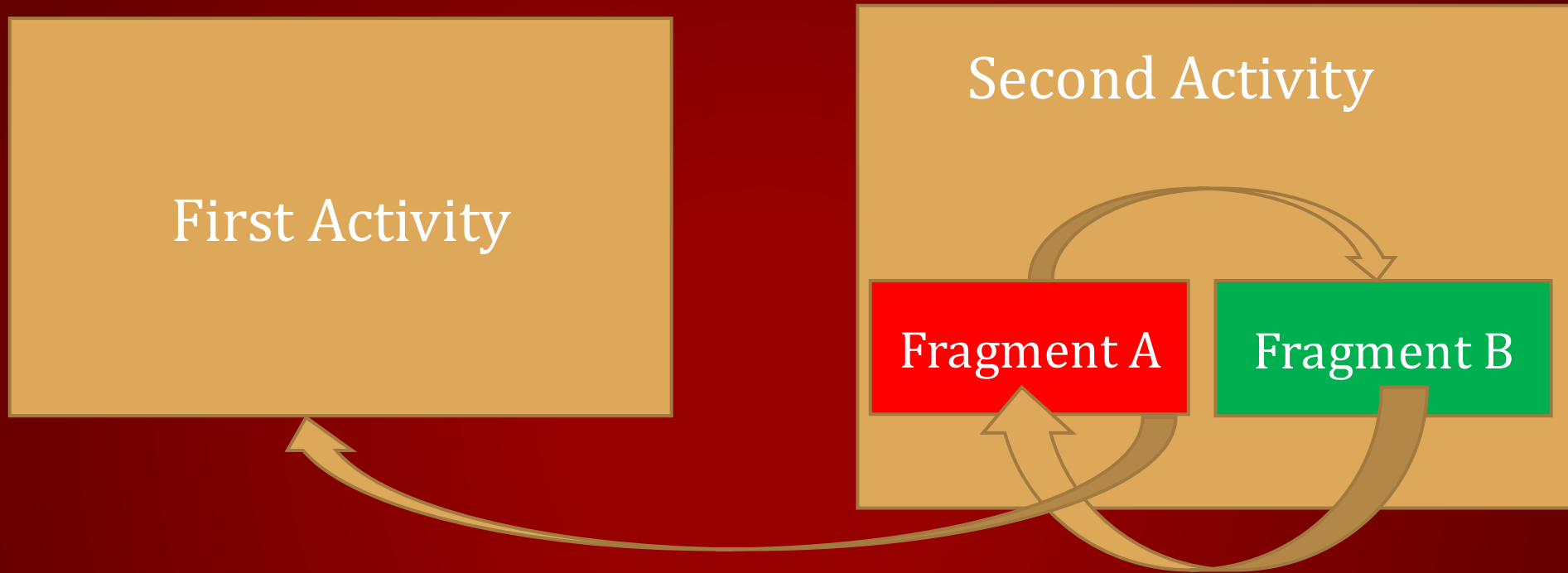
- ❑ Create an interface MyListener
 - ❑ Define a callback method
- ❑ Implement MyListener in Activity
- ❑ Pass data from FragmentA to Activity
 - ❑ `activity.add(x,y)`
- ❑ Pass data from Activity to FragmentB
 - ❑ `FragB.addTwoNumbers(x,y)`

❖ FRAGMENTS – ENABLING BACK BUTTON

- ❑ Activity responds to back button
- ❑ Fragments are not aware of back button
 - ❑ Fragments do not respond to back press



Solution: Use Back Stack



- ❑ If back stack concept is used
 - ❑ Then we can enable back press for fragments
 - ❑ We can switch back to our previous fragment

Transaction add/pop in back stack

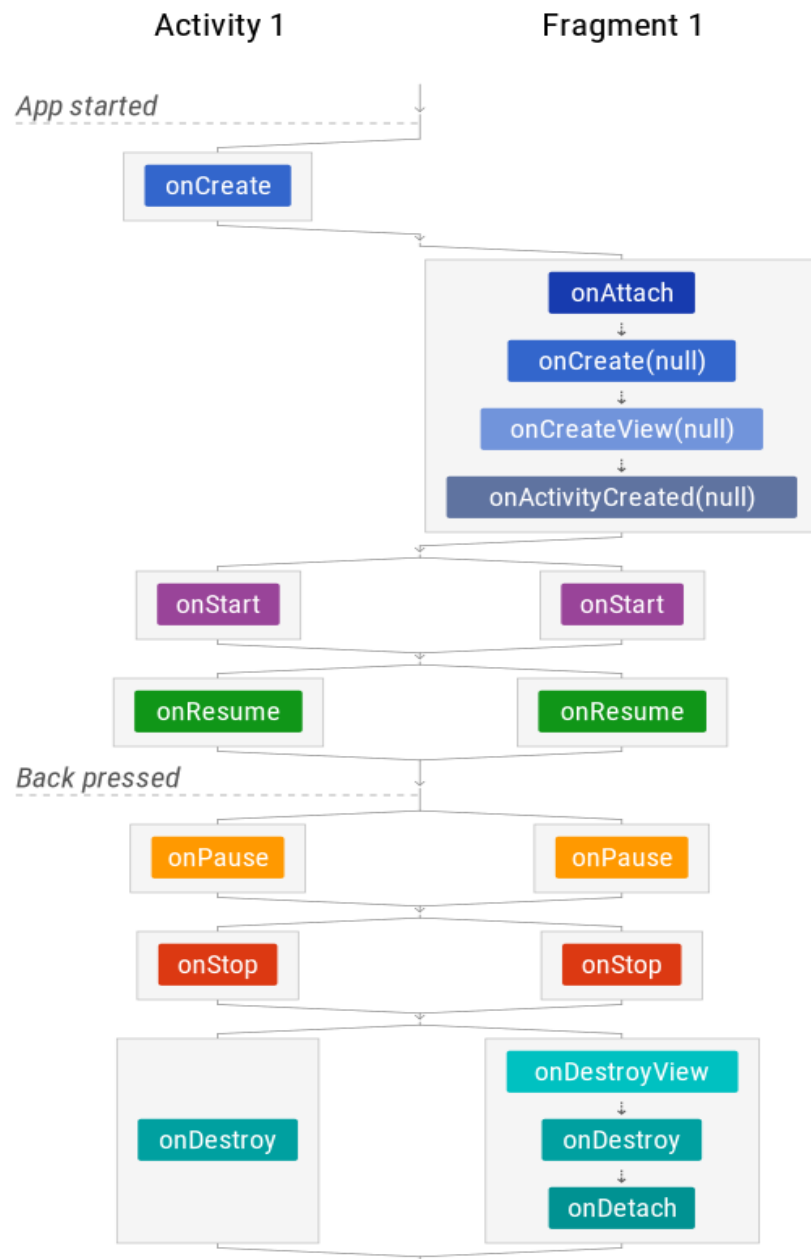
- ❑ To add a transaction in back stack
 - ❑ addToBackStack(null)
 - ❑ addtoBackStack(String TAG)
- ❑ pop a transaction in Back Stack
 - ❑ popBackStack()
 - ❑ popBackStack(String TAG, int FLAG)
 - ❑ int FLAG-0, POP_BACK_STACK_INCLUSIVE

```
//stacking
FragmentManager fm = getFragmentManager();
FragmentTransaction ft = fm.beginTransaction();
ft.addToBackStack("TAG");
ft.commit();
```

//Back to name state

```
FragmentManager fm = getFragmentManager();
fm.popBackStack("TAG", 0) //Return to the previous state of name
FragmentManager fm = getFragmentManager();
fm.popBackStack("TAG", FragmentManager.POP_BACK_STACK_INCLUSIVE)
```

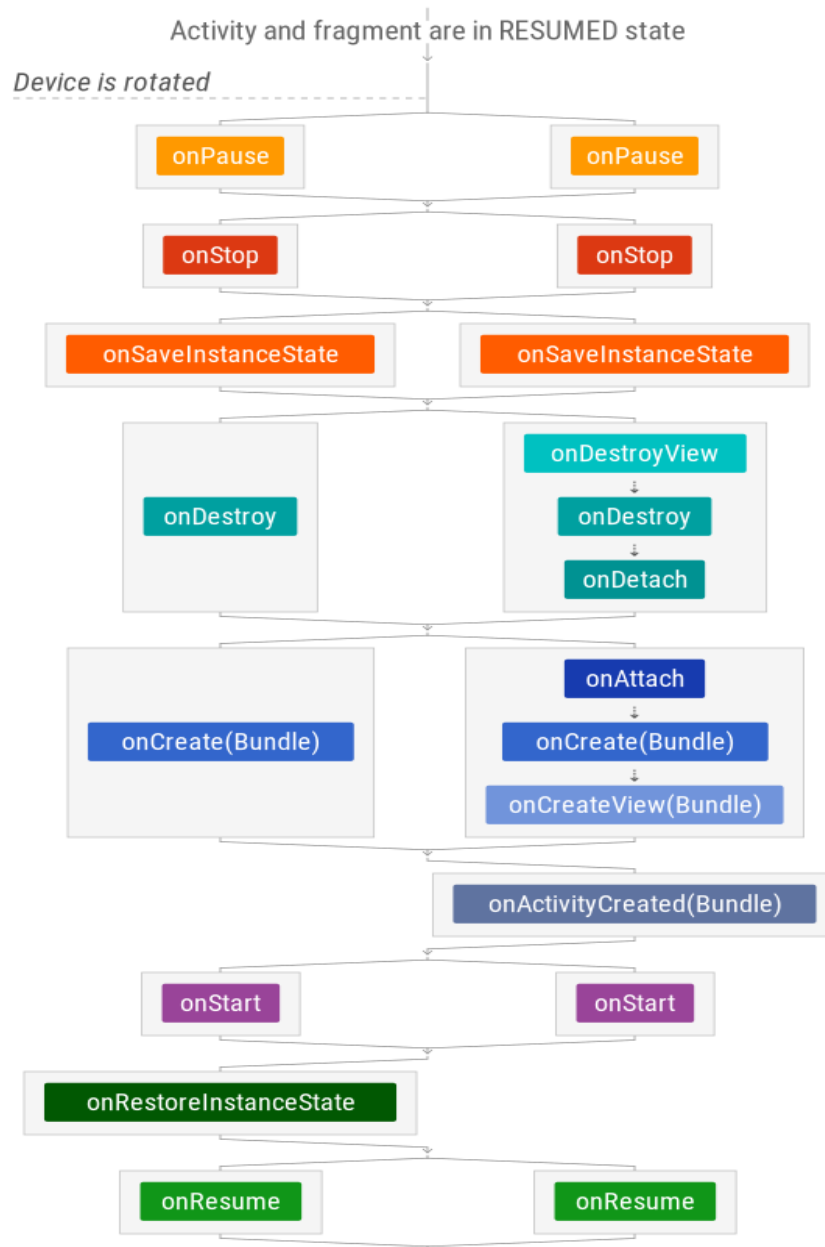
FRAGMENTS AND SCREEN ROTATION



ACTIVITY WITH FRAGMENT STATES- START TO FINISH

Activity 1

Fragment 1



ACTIVITY WITH FRAGMENT IS ROTATED